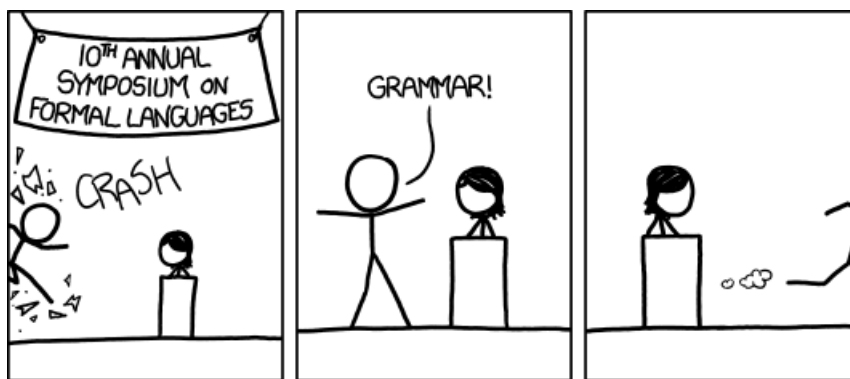


Chapitre 2

Langages



1 Mots

1.1 Alphabet et mots

Définition - Alphabet

Un **alphabet** est un ensemble fini dont les éléments sont appelés lettres.

EXEMPLES.

- En informatique on utilise souvent des alphabets de taille 2 : $\{0, 1\}$ ou $\{a, b\}$.
- En français, l'alphabet courant (sans accentuation) comporte 26 caractères,

$$\Sigma = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}.$$

- Le code génétique est écrit sur l'alphabet des nucléotides $\{A, T, G, C\}$ pour l'ADN.

Définition - Mot

Soit Σ un alphabet.

- Un mot u sur cet alphabet Σ est soit le mot vide ε , soit une suite finie $u_1 u_2 \dots u_n$ avec, pour tout $k \in [1, n]$, $u_k \in \Sigma$, c'est-à-dire, chaque u_k est un caractère de Σ .
- La longueur d'un mot u est l'entier noté $|u|$, qui vaut 0 si $u = \varepsilon$ et n si $u = u_1 u_2 \dots u_n$ avec, pour tout $k \in [1, n]$, $u_k \in \Sigma$.

Définition

- L'ensemble des mots de longueur n sur Σ est noté Σ^n . L'ensemble de tous les mots sur Σ est noté Σ^* . Par construction,

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n = \{\varepsilon\} \cup \bigcup_{n \in \mathbb{N}^*} \Sigma^n.$$

- L'ensemble de tous les mots non vides sur Σ est noté Σ^+ . Par construction,

$$\Sigma^+ = \bigcup_{n \in \mathbb{N}^*} \Sigma^n.$$

EXEMPLE. Si $\Sigma = \{a, b\}$ alors $\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

1.2 Concaténation

Définition - Concaténation de deux mots

La concaténation de deux mots $u = u_1 u_2 \dots u_n$ et $v = v_1 v_2 \dots v_p$ est le mot uv de longueur $n + p$ dont le i ème caractère est u_i si $i \leq n$ et v_{i-n} sinon.

u	v
-----	-----

EXEMPLE. La concaténation de $abba$ et de aab est $abbbaab$.

Théorème

La concaténation est une loi de composition interne associative sur Σ^* d'élément neutre ε .

Remarques

1. Grâce à l'associativité de la concaténation, on peut simplifier les écritures : on notera a^n pour le mot $aa \dots a$ où a apparaît n fois.
2. Il n'y a pas de symétrie pour la concaténation (sauf pour le mot vide), toutefois les éléments sont simplifiables à gauche comme à droite : $uv = uw$ ou $vu = wu$ entraîne $v = w$.
3. On a, par définition, l'additivité de la longueur pour la concaténation :
Soit $u, v \in \Sigma^*$ alors $|uv| = |u| + |v|$.

1.3 Préfixes, suffixes, facteurs

Définition

Soient $u, x \in \Sigma^*$, on dit que :

- x est un **préfixe** de u s'il existe un mot $y \in \Sigma^*$ tel que $u = xy$;
- x est un **suffixe** de u s'il existe un mot $y \in \Sigma^*$ tel que $u = yx$.
- x est un **facteur** de u s'il existe des mots $y, z \in \Sigma^*$ tels que $u = yxz$.
- Le mot x de longueur k est un **sous-mot** du mot $u = u_1 \dots u_n$, s'il existe des indices $1 \leq i_1 < i_2 < \dots < i_k \leq n$ tels que $x = u_{i_1} u_{i_2} \dots u_{i_k}$.

EXEMPLES.

- Les préfixes de MPI sont $\{\epsilon, M, MP, MPI\}$ et ses suffixes sont $\{\epsilon, I, PI, MPI\}$.
- Le mot $aoba$ est un facteur de $baobab$. Le mot bob est un sous mot de $baobab$ mais ce n'est pas un facteur.

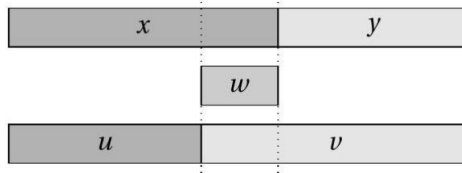
1.4 Quelques résultats combinatoires

Théorème (Lemme de Levi)

Soient x, y, u et $v \in \Sigma^*$ tels que $xy = uv$. Alors, il existe un unique mot w tel que l'une des conditions suivantes est réalisée

1. $x = uw$ et $v = wy$;
2. $u = xw$ et $y = wv$.

La compréhension de ce résultat est assez immédiate avec le schéma suivant (toujours faire un dessin pour ce type de preuves) :



DÉMONSTRATION.

Existence (par récurrence sur $|xy|$) :

- Si $|xy| = 0$ alors $x = y = u = v = \epsilon$ et donc dans ce cas $w = \epsilon$ convient.
- Soient donc $x, y \in \Sigma^*$ tels que $|xy| = n + 1$ avec $n \in \mathbb{N}$.
 - Si $x = \epsilon$: alors $w = u$ convient et on a $u = xw$ et $y = wv$.
 - Si $u = \epsilon$: alors $w = x$ convient et on a bien $x = uw$ et $v = wy$.
 - Sinon : il existe $a \in \Sigma$ et $x', u' \in \Sigma^*$ tels que $x = ax'$ et $u = au'$. Ainsi, l'égalité $xy = uv$ devient $ax'y = au'v$, d'où l'on peut extraire l'égalité suivante :

$$x'y = u'v$$

avec $|xy'| = |xy| - 1$.

Ainsi, par H.R il existe $w \in \Sigma^*$ tel que $\begin{cases} x' = u'w \\ v = wy \end{cases}$ ou $\begin{cases} u' = x'w \\ y = wv \end{cases}$

Ainsi, comme $x = ax', u = au'$, on a :

$$\begin{cases} x = au'w = uw \\ v = wy \end{cases} \quad \text{ou} \quad \begin{cases} u = ax'w = xw \\ y = wv \end{cases}$$

D'où le résultat voulu.

Unicité (sous réserve d'existence)

Notons $|w| = ||u| - |x||$.

- Si $|u| = |x|$ alors $w = \varepsilon$.
- Si $|u| > |x|$ alors w est l'unique suffixe de u de longueur $|u| - |x|$.
- Si $|u| < |x|$ alors w est l'unique suffixe de x de longueur $|x| - |u|$.

D'où l'existence et l'unicité d'un tel $w \in \Sigma^*$.

Corollaire

Soient x, y deux préfixes (respectivement suffixes) d'un mot $u \in \Sigma^*$. Alors,

x est un préfixe (respectivement suffixe) de y ou y est un préfixe (respectivement suffixe) de x .

DÉMONSTRATION.

En effet, $\exists u_1, u_2 \in \Sigma^*$ tels que $x = uu_1 = yu_2$ et donc d'après le théorème précédent :

$\exists w \in \Sigma^*$ tq. $x = yw$ (et donc y serait un préfixe de x) ou $y = xw$ (et donc x serait un préfixe de y).

On peut en déduire par exemple les résultats suivants :

Théorème

Soient x, y et z dans Σ^* tels que $xy = yz$ et $x \neq \varepsilon$.

Alors il existe deux mots u et v et un entier $k \in \mathbb{N}$ tels que :

$$x = uv, y = (uv)^k u = u(vu)^k, z = vu.$$

DÉMONSTRATION.

- Si $|x| > |y|$, on peut appliquer le lemme de Levi : il existe un mot $t \in \Sigma^*$ tel que $x = yt$ et $z = ty$. Dans ce cas le résultat est bien acquis avec $u = y, v = t$ et $k = 0$.
- Si $|x| \leq |y|$, raisonnons par induction sur la longueur de y .
 - Si $|y| = 1$ on a aussi $|x| = 1$ car $x \neq \varepsilon$ et dans ce cas $x = y = z$. Le résultat est acquis avec $u = y, v = \varepsilon$ et $k = 0$.
 - Si $|y| > 1$, supposons le résultat acquis pour tout mot de longueur inférieure et appliquons le lemme de Levi : il existe un mot t tel que $y = xt$ et $y = tz$. On a donc $xt = tz$ et puisque $x \neq \varepsilon$ on a $|t| < |y|$. Ainsi, par H.R., on peut affirmer l'existence de deux mots $u, v \in \Sigma^*$ tels que $x = uv, t = (uv)^k u = u(vu)^k$ et $z = vu$.
Il reste à calculer $y = xt = tz$ et obtenir $y = (uv)^{k+1} u = u(vu)^{k+1}$.

D'où l'hérédité.

Théorème

Soient x et y dans Σ^* tels que $xy = yx$, avec $x \neq \varepsilon$ et $y \neq \varepsilon$. Alors,

il existe un mot $u \in \Sigma^*$ et deux entiers i et j tels que $x = u^i$ et $y = u^j$.

DÉMONSTRATION. (par induction sur $|xy|$)

- Si $|xy| = 2$ alors $|x| = |y| = 1$ et $x = y$. Le résultat est acquis en posant $u = x = y$ et $i = j = 1$.
- Si $|xy| > 2$, supposons le résultat acquis pour tout mot de longueur inférieure et appliquons le théorème précédent : il existe deux mots $u, v \in \Sigma^*$ et un entier $k \in \mathbb{N}$ tels que $x = uv = vu$ et $y = (uv)^k u = u(vu)^k$. Supposons sans perte de qualité que nous nous situons dans le premier cas.
Ainsi, on obtient $yw = wy$ avec $|wy| < |xy|$ ($y \neq \varepsilon$).
Par H.R., $\exists u \in \Sigma^*, (i, j) \in \mathbb{N}^2$ tq. $y = u^i$ et $w = u^j$.
Or $x = yw$ donc $x = u^{i+j}$.

d'où l'hérédité.

2 Langages

Définition - Langage

Un langage sur l'alphabet Σ est une partie de Σ^* .

EXEMPLES. Voici quelques exemples de langages sur l'alphabet $\Sigma = \{a, b\}$:

- $\{a, ab, abba, ababbab\}$
- $\{a, b\}^*$
- l'ensemble des mots se terminant par ba
- l'ensemble des mots de longueur paire
- l'ensemble des mots palindromes (égaux à leur symétrique miroir).

⇒ Les langages étant des ensembles, ils sont susceptibles d'être soumis aux opérations ensemblistes : réunion, intersection, passage au complémentaire, différence symétrique. On ajoute à cette liste l'opération de concaténation et la puissance de langages.

Définition - Concaténation de langages

Soient L_1, L_2 deux langages sur l'alphabet Σ . La concaténation de L_1 et L_2 est le langage $L_1 L_2$ défini par

$$L_1 L_2 = \{uv, u \in L_1, v \in L_2\}.$$

EXEMPLES.

1. Si L_1 est le langage des mots comprenant uniquement la lettre a et si L_2 est le langage des mots comprenant uniquement la lettre b , alors $L_1 L_2$ est le langage des mots s'écrivant comme une succession de a puis une succession de b .
2. Soit L_1 l'ensemble des mots de longueur paire et L_2 l'ensemble des mots de longueur impaire. Que représente $L_1 L_2$? Il représente l'ensemble des mots de longueur impaire.
3. Attention : $L \cdot \emptyset = \emptyset \cdot L = \emptyset$

Définition - Étoile de Kleene

Soit L un langage.

Pour un entier $n \in \mathbb{N}$, on définit L^n par :

- $L^0 = \{\epsilon\}$
- $L^{n+1} = LL^n = L^n L$ si $n \in \mathbb{N}$.

On définit l'**étoile de Kleene** de L par $L^* = \bigcup_{n \in \mathbb{N}} L^n$ et $L^+ = \bigcup_{n \in \mathbb{N}^*} L^n$.

⚠ Méfiez-vous !

Attention, il ne faut pas confondre L^2 et $\{u^2 \mid u \in L\}$. En effet, pour $L = \{ab, ba\}$, on a :

$$L^2 = \{abab, abba, baab, baba\} \text{ alors que } \{u^2 \mid u \in L\} = \{abab, baba\}$$

3 Langages réguliers et expressions régulières

Définition - Langage régulier

Soit Σ un alphabet. L'ensemble des langages réguliers sur Σ est défini inductivement de la façon suivante :

- $\emptyset$ est régulier.
- Pour tout $a \in \Sigma$, $\{a\}$ est régulier.
- Si L_1 et L_2 sont deux langages réguliers alors $L_1 \cup L_2$ ainsi que $L_1 L_2$ sont aussi réguliers.
- Si L est régulier alors L^* l'est aussi.

On note $\text{Reg}(\Sigma)$, l'ensemble des langages réguliers sur Σ .

EXEMPLES.

1. Montrer que $\{\epsilon\}$ est régulier.

On a bien $\emptyset^* = \{\epsilon\}$

d'où le résultat voulu.

2. Montrer que tout ensemble fini est régulier.

Soit $E = \{e_1, \dots, e_n\}$ un ensemble fini quelconque, tel que $\forall i \in \llbracket 1, n \rrbracket, e_i \in \Sigma$. Alors, $\forall i \in \llbracket 1, n \rrbracket, \{e_i\}$ est régulier.

Enfin,

$$E = \bigcup_{1 \leq i \leq n} \{e_i\}$$

or par récurrence immédiate, une union dénombrable de langage régulier est un langage régulier.

D'où le résultat voulu.

Définition - Expression régulière

Un expression régulière est définie inductivement par :

- $\emptyset$ et ϵ sont des expressions régulières.
- Pour tout $a \in \Sigma$, a est une expression régulière.
- Si E_1 et E_2 sont deux expressions régulières alors $E_1 | E_2$ ainsi que $E_1 E_2$ sont aussi régulières.
- Si E est une expression régulière alors E^* l'est aussi.

Les expressions régulières permettent d'avoir une description des langages réguliers.

Définition

Soit E un expression régulière sur un alphabet Σ , alors on définit inductivement le langage associé à E (noté $\mathcal{L}(E)$) de la manière suivante :

- $\mathcal{L}(\emptyset) = \emptyset$.
- $\mathcal{L}(\epsilon) = \{\epsilon\}$.
- $\forall a \in \Sigma, \mathcal{L}(a) = \{a\}$.
- $\mathcal{L}(E_1 | E_2) = \mathcal{L}(E_1) \cup \mathcal{L}(E_2)$ et $\mathcal{L}(E_1 E_2) = \mathcal{L}(E_1) \mathcal{L}(E_2)$.
- $\mathcal{L}(E^*) = \mathcal{L}(E)^*$

EXEMPLES.

1. Quel est le langage associé à l'expression : $0^* 10^* 10^* 10^*$? Il s'agit des successions de 0 et de 1 contenant exactement 3 fois 1.
2. Donner une expression régulière correspondant à l'écriture en base deux d'un entier sans 0 non significatif : $1(1|0)^* + \epsilon$

4 Automates finis déterministes

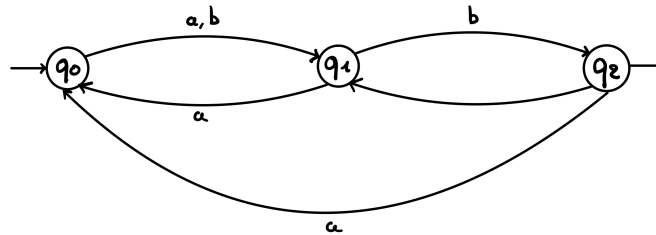
4.1 Définitions et premiers exemples

Définition - Automate fini déterministe

Un automate fini déterministe est un quintuplet $(\Sigma, Q, q_0, F, \delta)$ avec Σ un alphabet, Q un ensemble d'états, $q_0 \in Q$ l'état initial, $F \subset Q$ l'ensemble des états terminaux ou acceptant et δ une fonction partiellement définie qui va de $Q \times \Sigma$ dans Q .

EXEMPLE. On prend l'automate $A = (\{a, b\}, \{q_0, q_1, q_2\}, q_0, \{q_2\}, \delta)$ et un mot $m = aabba$.

L'automate lit les lettres du mots et fini dans l'état q_1 . Le mot n'est donc pas accepté par la machine car elle n'est pas dans un état acceptant.



⚠ Un mot est accepté par A si et seulement si la lecture de ce mot dans A aboutit à un état acceptant. Ici m n'est pas reconnu alors que $m' = mb$ l'est. Définissons formellement le fait qu'un mot soit accepté ou non.

Définition - Fonction de transition étendue

Soit A un automate et $m \in \Sigma^*$ on définit δ^* la fonction de transition étendue $\delta^* : Q \times \Sigma^* \rightarrow Q$ de la manière suivante :

$$\begin{cases} \forall q \in Q, \delta^*(q, \epsilon) = q \\ \forall q \in Q, \forall a \in \Sigma, \forall m \in \Sigma^*, \delta^*(q, am) = \delta^*(\delta(q, a), m) \end{cases}$$

Définition - Extension de l'acceptation

On dit que $m \in \Sigma^*$ est accepté par A si $\delta^*(q_0, m) \in F$.

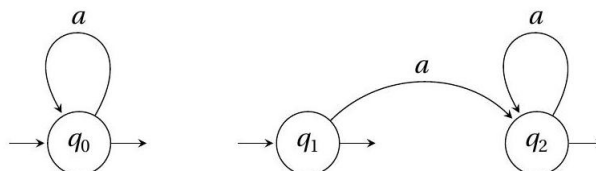
On remarque que si la lettre a bloqué, le mot n'est pas accepté.

Définition - Langage des mots acceptés et langage reconnaissable

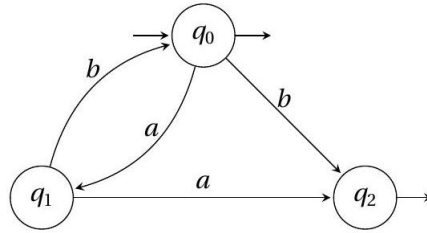
- Étant donné un automate A , on note $L(A)$ le langage des mots acceptés par A (qu'on appelle aussi *langage reconnu*).
- Un langage L est dit *reconnaissable* s'il existe un automate qui le reconnaît.

EXEMPLES.

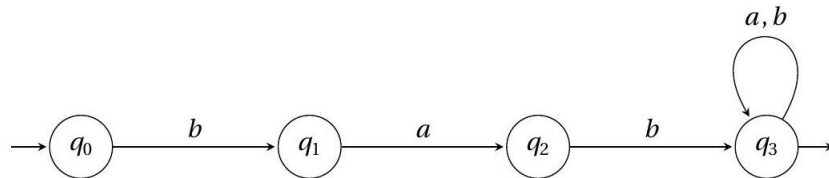
1. Les deux automates suivants sur l'alphabet $\Sigma = \{a\}$ reconnaissent Σ^*



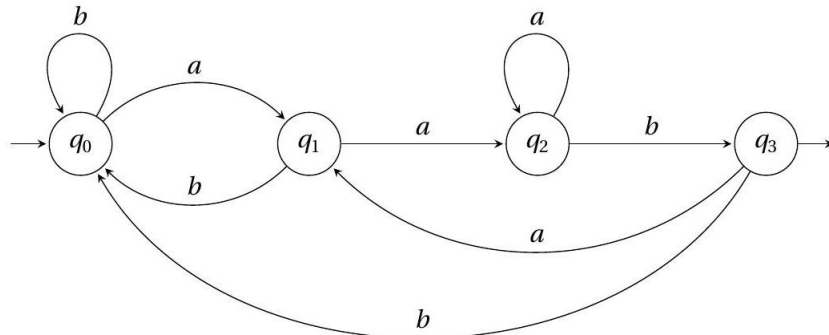
2. L'automate suivant reconnaît le langage $\{bab\}\Sigma^*$, c'est-à-dire l'ensemble des mots commençant par bab .



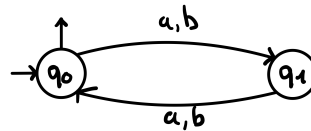
3. L'automate suivant reconnaît le langage $\{ab\}^*\{aa, b, \varepsilon\}$.



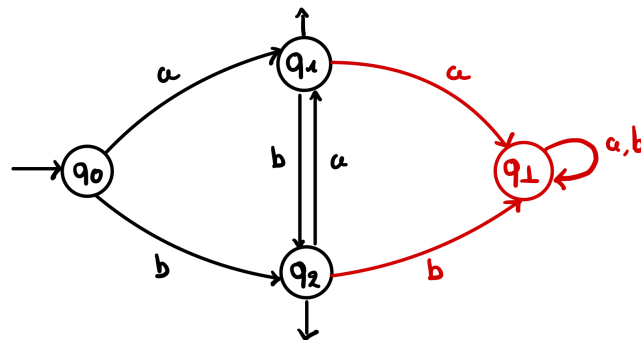
4. Quel est le langage reconnu par l'automate suivant? Les mots qui se terminent par aab .



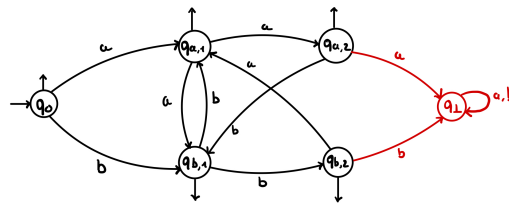
5. Construire un automate qui reconnaît le langage des mots de longueur paire sur l'alphabet $\{a, b\}$.



6. Construire un automate qui reconnaît le langage des mots sur l'alphabet $\{a, b\}$ où deux lettres identiques ne se suivent jamais : $ababa$ est dans le langage mais pas $aabab$.



7. Construire un automate qui reconnait le langage des mots sur l'alphabet $\{a, b\}$ où une occurrence d'une même lettre ne peut pas apparaître plus de deux fois de suite.



⇒ Ne pas hésiter, en construisant, à traduire l'information : « je suis dans tel état signifie que j'ai lu ... »

⚠ Remarque importante

Il est facile de mettre en évidence l'existence de langages non reconnaissables. En effet, Σ^* est infini donc $P(\Sigma^*)$ est indénombrable c'est à dire l'ensemble des langages est non dénombrable. Or il existe un nombre dénombrable d'automates : car si n est un entier naturel il y a un nombre fini d'automate dont les états sont l'ensemble $\llbracket 0, n-1 \rrbracket$. Or une union dénombrable d'ensembles finis est dénombrable.

4.2 Implémentation

Les états sont numéroté de 0 à $n-1$ avec n le nombre d'états. On considère que 0 est l'état initial.

```
1 type aut = {taille: int ; final: bool array; delta: int->char->int}
2 exception Blocage
```

où l'état initial est toujours l'état 0, `finalaux.(i)` vaut `true` si, et seulement si, l'état `i` est final. `delta` est une fonction qui n'est pas nécessairement définie pour tous les états et lettres mais qui lance l'exception `Blocage` si l'automate est bloqué (on pourrait également utiliser un type option !). On implémente également les fonctions suivantes :

```
1 let delta_star (a : aut) (m : string) : int =
2   let n = String.length m in
3   let rec aux (q : int) (i : int) : int =
4     if i = n then q
5     else aux (a.delta q m.[i]) (i+1)
6   in aux 0 0
7
8 let accept a m =
9   try
10    a.final (delta_star a m)
11  with Blocage -> false
```

4.3 Propriété sur les automates finis déterministes auxquelles on peut se ramener

4.3.1 Complétude

Définition - Automate complet

Un automate $A = (\Sigma, Q, q_0, F, \delta)$ est dit **complet** si δ est définie sur $\Sigma \times Q$ de manière totale.

Théorème

Tout automate fini déterministe est équivalent à un automate complet dans le sens où ils reconnaissent tous les deux le même langage.

DÉMONSTRATION.

Soit $A = (\Sigma, Q, q_0, F, \delta)$. On définit un automate $A_c = (\Sigma, Q \cup \{q_\perp\}, q_0, F, \delta_c)$ avec δ_c tel que $\forall a \in \Sigma, \forall q \in Q$ si $\delta(q, a)$ est défini alors $\delta_c(q, a) = \delta(q, a)$ sinon $\delta_c(q, a) = q_\perp$ et $\forall a \in \Sigma, \delta_c(q_\perp, a) = q_\perp$ (c'est la création de l'état *poubelle* !). Il est donc complet par construction.

Reste à prouver que $L(A) = L(A_c)$:

Soit $m \in L(A)$. Ainsi $\delta^*(q_0, m)$ est bien défini et $\delta^*(q_0, m) \in F$ donc $\delta^*(q_0, m) = \delta_c^*(q_0, m) \in F$. Donc $m \in L(A_c)$.

Soit $m \in L(A_c)$. On note $m = m_1 \dots m_n$. On sait donc que $\delta_c^*(q_0, m) \in F$.

On pose $q_i = \delta_c^*(q_0, m_1 \dots m_i)$. On remarque que pour tout i , $q_i \neq q_\perp$ car s'il existait i tel que $q_i = q_\perp$ on aurait pour tout $j \geq i$ $q_j = q_\perp$ or $q_n = \delta_c^*(q_0, m) \in F$ donc $q_n \neq q_\perp$.

Ainsi $\delta_c^*(q_0, m) = \delta^*(q_0, m)$ car on ne passe jamais par $q_\perp$ donc $\delta^*(q_0, m)$ existe et appartient à F donc $m \in L(A)$.

Corollaire

La classe des langages reconnaissables est stable par complémentaire.

DÉMONSTRATION.

Soit $A = (\Sigma, Q, q_0, F, \delta)$. On pose $L = \mathcal{L}(A)$ et $L' = \Sigma^* \setminus L(A)$.

On suppose A complet.

Ainsi $\forall m \in \Sigma^*$, $\delta^*(q_0, m)$ est bien défini et $(m \in L(A) \Leftrightarrow \delta^*(q_0, m) \in F)$ et $(m \notin L(A) \Leftrightarrow \delta^*(q_0, m) \notin F)$.

On pose $A' = (\Sigma, Q, q_0, Q \setminus F, \delta)$ et on a alors $m \in L' \Leftrightarrow m \notin L = \mathcal{L}(A) \Leftrightarrow \delta^*(q_0, m) \in Q \setminus F \Leftrightarrow m \in \mathcal{L}(A')$

D'où $L' = \mathcal{L}(A')$ avec A' un automate fini déterministe.

D'où L' reconnaissable.

4.3.2 Émondage

Définition - « Être dans tous ses états »

Soit $A = (\Sigma, Q, q_0, F, \delta)$ un automate.

- Un état $q \in Q$ est dit **accessible** si et seulement si il existe $u \in \Sigma^*$ tel que $\delta^*(q_0, u) = q$.
- Un état $q \in Q$ est dit **co-accessible** si et seulement si il existe $u \in \Sigma^*$ et $f \in F$ tel que $\delta^*(q, u) = f$.
- Un état $q \in Q$ est dit **utile** s'il est accessible et co-accessible (il existe un chemin de q_0 vers un état de F qui passe par q).

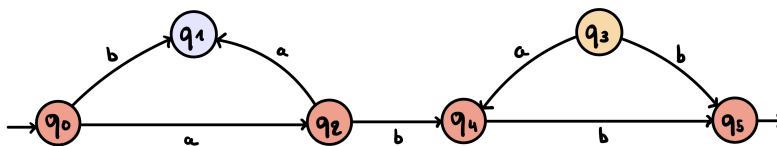
Définition - Automate émondé

Un automate est dit **émondé** quand tous ses états sont utiles.

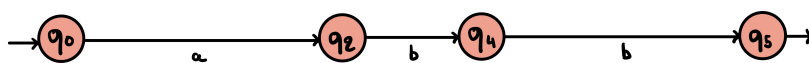
Remarque

On remarque qu'un état puits n'est pas co-accessible donc en général on n'a pas les deux propriétés (complet et émondé) simultanément.

EXEMPLE. On colorie en bleu les états accessibles, en orange les co-accessibles et en rouge les états utiles de l'automate A ci-dessous.



On dresse alors l'automate émondé issu de celui ci-dessus :



Théorème

Tout automate fini déterministe qui reconnaît un langage non vide est équivalent à un automate émondé.

DÉMONSTRATION.

On retire tous les états non-utiles de l'automate. Ainsi on note A l'automate d'origine et A' l'automate émondé. D'où $A = (\Sigma, Q, q_0, F, \delta)$ et $A' = (\Sigma, Q', q_0, F', \delta_{Q'})$.

Soit $m \in L(A')$. Alors $\delta_{Q'}(q_0, m) \in F' \subset F$ donc $\delta^*(q_0, m) \in F$ c'est à dire $m \in L(A)$.

Soit $m \in L(A)$. On pose $m = m_1 \dots m_n$ et $\forall i, q_i = \delta^*(q_0, m_1 \dots m_i)$ chaque q_i est accessible mais, aussi co-accessible car on peut on atteindre un état final depuis q_i avec le mot $m_{i+1} \dots m_n$. Ainsi le chemin $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_n$ existe dans A' avec $q_n \in F'$ car q_n est accessible donc $m \in L(A')$.

4.4 Automate produit

Définition - Automate produit

Soient $A_1 = (\Sigma, Q_1, q_0^1, F_1, \delta_1)$ et $A_2 = (\Sigma, Q_2, q_0^2, F_2, \delta_2)$. On définit l'automate produit $A_1 \otimes A_2$ par :

$$A_1 \otimes A_2 = (\Sigma, Q_1 \times Q_2, (q_0^1, q_0^2), F, \delta)$$

avec $\forall a \in \Sigma, \forall q_1 \in Q_1, \forall q_2 \in Q_2, \delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$.

Théorème

La classe des langages reconnaissables est stable par union et par intersection.

DÉMONSTRATION.

Soient $A_1 = (\Sigma, Q_1, q_0^1, F_1, \delta_1)$ et $A_2 = (\Sigma, Q_2, q_0^2, F_2, \delta_2)$. On pose $A_1 \otimes A_2 = (\Sigma, Q_1 \times Q_2, (q_0^1, q_0^2), F, \delta)$.

On peut montrer par récurrence sur $|m|$ que :

$$\forall m \in \Sigma^*, q_1 \in Q_1, q_2 \in Q_2, \delta^*((q_1, q_2), m) = (\delta_1^*(q_1, m), \delta_2^*(q_2, m))$$

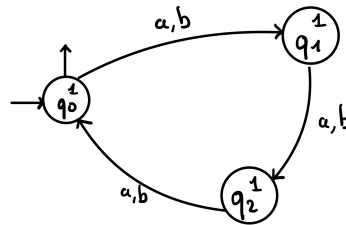
On a alors $m \in L(A_1 \otimes A_2) \iff \delta^*((q_0^1, q_0^2), m) = (\delta_1^*(q_0^1, m), \delta_2^*(q_0^2, m)) \in F$.

Ainsi si $F = F_1 \times F_2$ on aura $L(A_1 \otimes A_2) = L(A_1) \cap L(A_2)$ et si $F = (F_1 \times Q_2) \cup (F_2 \times Q_1)$ on aura $L(A_1 \otimes A_2) = L(A_1) \cup L(A_2)$.

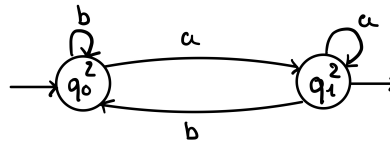
Ainsi le langage $L(A_1) \cap L(A_2)$ est reconnaissable car il existe un automate $A_1 \otimes A_2$ tel $L(A_1) \cap L(A_2) = L(A_1 \otimes A_2)$.

EXEMPLES.

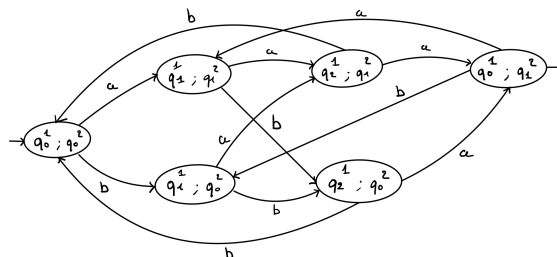
1. Donnons un automate qui reconnaît les mots ayant une longueur multiple de 3.



2. Donnons un automate qui reconnaît les mots de $\Sigma = \{a, b\}$ terminant par a .



3. Donnons un automate qui reconnaît les mots dont la longueur est un multiple de 3 et terminant par a .



5 Automates finis non-déterministes

Nous allons maintenant considérer des automates dans lesquels à la lecture d'un mot, il y a plusieurs manières possibles de lire un tel mot. Un mot sera accepté ssi l'une des manières de le lire permet de passer de l'état initial à un état final. Prenons un exemple simple : considérons le langage des mots terminant par a :

5.1 Non-déterminisme

On va donc s'autoriser à avoir une fonction de transition qui, à la lecture d'une lettre sur un état donné, permet d'accéder à plusieurs états différents. On relâche donc l'hypothèse de lecture unique d'un mot dans le graphe de l'automate. Ainsi, on relâchera aussi le fait d'avoir un seul état initial : à la lecture du mot vide ϵ , on peut aussi se retrouver dans plusieurs états différents.

Définition - Automate fini non-déterministe

Un automate fini non-déterministe est un quintuplet $A = (\Sigma, Q, I, F, \delta)$ avec $I \subset Q$ un ensemble d'états initiaux et $\delta : Q \times \Sigma \rightarrow P(Q)$ définie de manière partielle.

On retrouve également une définition totalement équivalente :

Définition - Automate fini non-déterministe (v2)

Un automate fini non déterministe est un quintuplet $A = (\Sigma, Q, I, F, R)$ où R est une relation d'ordre sur $Q \times \Sigma \times Q$ où $R(q, a, q')$ est vrai si et seulement l'automate peut passer de l'état q à q' en lisant la lettre a si et seulement si $q' \in \delta(q, a)$.

On définit alors (de la manière qu'au paragraphe précédent) la fonction de transition étendue dans ce contexte : plutôt que d'avoir comme image un état, la fonction aura pour image un ensemble d'états.

Définition - Fonction de transition étendue

On définit la fonction de transition étendue $\delta^* : Q \times \Sigma^* \rightarrow P(Q)$ par :

- $\delta^*(q, \epsilon) = \{q\}$
- $\forall q \in Q, \forall m = au \in \Sigma^*, \delta^*(q, au) = \bigcup_{q' \in \delta(q, a)} \delta^*(q', u)$.
-

$$\delta^* : \begin{cases} Q \times \Sigma^* \rightarrow P(Q) \\ (q, m) \rightarrow \begin{cases} \{q\} & \text{si } m = \epsilon \\ \bigcup_{q' \in \delta(q, a)} \delta^*(q', m') & \text{si } m = am' \end{cases} \end{cases}$$

Définition - Langage reconnu par un AFND

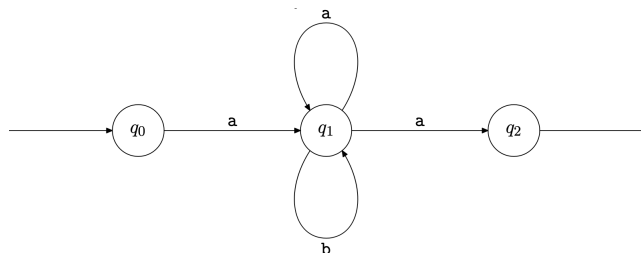
Soit un AFND $A = (Q, I, F, \delta)$ sur l'alphabet Σ et δ^* la fonction de transition étendue.

- Un mot $u \in \Sigma^*$ est dit reconnu par A s'il existe $q_0 \in I$ tel que $\delta^*(q_0, u) \cap F \neq \emptyset$.
- Le langage reconnu par A , noté $\mathcal{L}(A)$, est l'ensemble des mots reconnus par A .

Moins formellement, u est accepté par l'automate si parmi tous les chemins possibles lors de la lecture de u , il en existe un d'un état initial vers un état final.

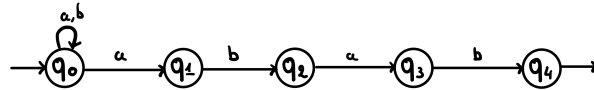
EXEMPLES.

1. Quel est le langage reconnu par l'automate suivant :



$$\Rightarrow \mathcal{L}(A) = \mathcal{L}(a\Sigma^*a)$$

2. Donner un automate non déterministe qui reconnaît le langage des mots terminant par $abab$:

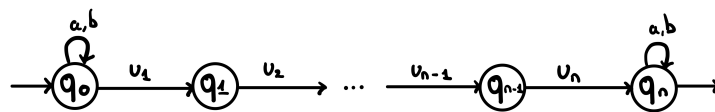


5.2 Pourquoi deux modèles?

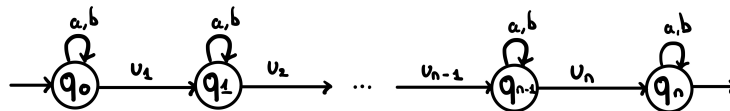
Un avantage des automates finis non déterministes sur les automates finis déterministes est **leur simplicité de manipulation** : il est beaucoup plus facile de concevoir un automate non déterministe reconnaissant un langage donné mais il est aussi plus facile de les manipuler dans des démonstrations formelles :

EXEMPLES.

1. Soit $u = u_1 \dots u_n$ un mot donné. Donner un AFND qui reconnaît le langage des mots dont un facteur est u . Même question avec u un sous-mot. On a



et



2. Soit A_1 un AFND qui reconnaît un langage L_1 et soit A_2 un AFND qui reconnaît un langage L_2 . Trouver un automate A qui reconnaît $L_1 L_2$ (auss appelé *automate de concaténation*).

Soient $A_1 = (\Sigma, Q_1, I_1, F_1, \delta_1)$ et $A_2 = (\Sigma, Q_2, I_2, F_2, \delta_2)$ tels que $\mathcal{L}(A_1) = L_1$ et $\mathcal{L}(A_2) = L_2$.

On construit

$$A_{1,2} = (\Sigma, Q_1 \cup Q_2, I_1, F_2, \delta)$$

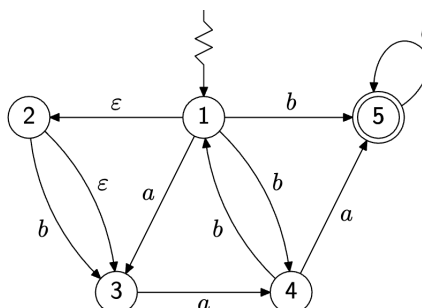
avec :

- $\forall q \in Q_1, \forall a \in \Sigma,$
 - si $\delta_1(q, a) \notin F_1$ alors $\delta(q, a) = \{\delta_1(q, a)\}$
 - si $\delta_1(q, a) \in F_1$ alors $\delta(q, a) = \{\delta_1(q, a)\} \cup \{q_i\}$
- $\forall q \in Q_2, \forall a \in \Sigma, \delta(q, a) = \{\delta_2(q, a)\}$

5.3 Epsilon-transitions

Pour rajouter au non déterministe, on peut même étendre la définition d'automate non déterministe en autorisant l'utilisation d' ϵ -transitions, aussi appelées transitions instantanées qui permettent de passer d'un état à un autre sans lire de lettre (on rappelle que ϵ n'est pas une lettre!!).

EXEMPLE. baa et aa sont par exemple reconnus par cet automate :



Définition - AFND

Soit Σ un alphabet fini. Un **automate fini non déterministe** est un quintuplet $(Q, I, F, \delta, \varphi)$ où

- Q , un ensemble fini dont les éléments sont appelés états de l'automate
- $I \subset Q$, appelé **ensemble des états initiaux**
- $F \subset Q$, appelé ensemble des états finaux (ou acceptants)
- δ une application de $Q \times \Sigma$ dans $\mathcal{P}(Q)$, appelée fonction de transition.
- φ est une application de Q dans $\mathcal{P}(Q)$ qui associe à chaque état l'ensemble des états auxquels il peut accéder par une ϵ -transition.

Définition - Fermeture transitive

Soit Σ un alphabet fini. Soit A un automate fini non déterministe défini par le quintuplet $(Q, I, F, \delta, \varphi)$ et soit $Q' \subset Q$. On définit la **clôture réflexive transitive** de Q' par ϵ -transition, notée $\kappa(Q')$ par la plus petite partie de Q contenant Q' et stable par ϵ -transition c'est-à-dire une partie P vérifiant :

$$\bigcup_{q' \in P} \varphi(q') \subset P$$

➤ Intuitivement, $\kappa(Q')$ désigne l'ensemble de tous les états (dont ceux de Q') qui peuvent être atteints à partir de Q' par un nombre fini quelconque d' ϵ -transitions.

Théorème

Soit Σ un alphabet fini. Soit A un automate fini non déterministe défini par le quintuplet $(Q, I, F, \delta, \varphi)$ et soit $Q' \subset Q$. Alors,

La suite d'ensembles définie par $K_0 = Q'$ et $\forall n \in \mathbb{N}, K_{n+1} = K_n \cup \left(\bigcup_{q' \in K_n} \varphi(q') \right)$ est stationnaire de limite $\kappa(Q')$.

DÉMONSTRATION.

Notons, pour $n \in \mathbb{N}$, $u_n = \text{card}(K_n)$. Ainsi, (u_n) est par construction croissante. Elle est de plus majorée : en effet, $u_n \leq |Q|$. Elle admet donc une limite $l \in \mathbb{N}$. Enfin, $(u_n) \in \mathbb{N}^{\mathbb{N}}$ donc elle devient stationnaire à partir d'un certain rang.

Ainsi, comme $(K_n)_n$ est croissante par inclusion, l'égalité des cardinaux à partir d'un certain rang nous garantit le fait que $(K_n)_n$ est stationnaire. Notons K_l sa limite. Reste à montrer que $K_l = \kappa(Q')$.

➤ Montrons d'abord que $K_l \subset \kappa(Q')$:

- Si $p \in \mathbb{N}$, notons H_p : « $K_p \subset \kappa(Q')$ »
- $p = 0$: par définition, $K_0 = Q' \subset \kappa(Q')$

H_0 vraie.

- $H_p \Rightarrow H_{p+1}$: Soit $q \in K_{p+1}$.
 - Si $q \in K_p$: par H.R on a le résultat recherché.
 - Sinon : nécessairement, $q \in K_{p+1} \setminus K_p$ donc il existe $q' \in K_p$ tq. $q \in \varphi(q')$.
Or, par H.R $K_p \subset \kappa(Q')$ et par def, $\kappa(Q')$ est la plus petite partie de Q contenant Q' stable par ϵ -transitions. q fait donc nécessairement parti de $\kappa(Q')$ sinon cette partie ne serait pas stable.

H_{p+1} vraie.

Ce qui clôt la récurrence.

Le résultat étant vrai pour tout K_n , il est alors vrai pour sa limite K_l (on rappelle que c'est une suite stationnaire).

➤ Montrons ensuite que $\kappa(Q') \subset K_l$: Pour cela, même pas besoin de récurrence ! En effet, $\kappa(Q')$ est par définition la plus petite partie de Q contenant Q' stable par ϵ -transitions (i.e par la fonction φ). Il suffit donc de montrer que $Q' \subset K_l$ et que K_l est stable par ϵ -transitions.

— On a immédiatement que $Q' \subset K_l$ car $Q' = K_0 \subset K_l$.

— Soit $q \in K_l$. Alors, par définition, $\varphi(q) \subset K_{l+1} = K_l$ (car $(K_n)_n$ est stationnaire). D'où la stabilité par φ .

Ainsi, K_l respecte les mêmes propriétés de $\kappa(Q')$, $\kappa(Q')$ est le plus petit ensemble qui vérifie ces propriétés. D'où

D'où l'égalité des 2 ensembles.

D'où le résultat voulu.

Les définitions de calculs et de langage reconnus s'adaptent alors facilement :

Définition - Fonction de transition étendue aux mots

Soit un automate non déterministe $(Q, I, F, \delta, \varphi)$ sur l'alphabet Σ . La fonction de transition δ^* étendue aux mots est la fonction $\delta^* : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$ définie récursivement par :

- pour tout $q \in Q, \delta^*(q, \epsilon) = \kappa(\{q\})$.
- pour tout $q \in Q, x = ua \in \Sigma^*, \delta^*(q, x) = \kappa\left(\bigcup_{q' \in \delta^*(q, u)} \delta(q', a)\right)$.

Définition - Reconnaître ou ne pas reconnaître telle est la question

Soit un AFND $A = (Q, I, F, \delta, \varphi)$ sur l'alphabet Σ et δ^* la fonction de transition étendue.

- Un mot $u \in \Sigma^*$ est dit reconnu par A s'il existe $q_0 \in I$ tel que $\delta^*(q_0, u) \cap F \neq \emptyset$.
- Le langage reconnu par A , noté $\mathcal{L}(A)$, est l'ensemble des mots reconnus par A .

➤ Il est évident que l'ensemble des langages reconnus par AFD est inclus dans celui reconnus par AFND. Qu'en est-il de la réciproque? Existe-t-il des langages reconnus par un AFND pour lesquels on ne peut pas construire d'AFD?

5.4 Élimination des ϵ -transitions

On considère un AFND $\mathcal{A} = (Q, I, F, \delta, \varphi)$.

Théorème

Tout AFND avec ϵ -transition(s) est équivalent à un AFND n'en comportant aucune.

DÉMONSTRATION. (admise)

5.4.1 Méthode 1

Élimination des ϵ -transitions - Méthode 1

Soient $q, q' \in Q$ tq. $q' \in \varphi(q)$.

Alors,

Toute transition entrante en q est dupliquée en une transition entrante en q' .

Remarque

- Toute transition entrante au sens littéral, i.e on considère également la flèche d'entrée. Donc **si $q \in I$, alors q' doit également être un état initial.**

5.4.2 Méthode 2

Élimination des ϵ -transitions - Méthode 2

Soient $q, q' \in Q$ tq. $q' \in \varphi(q)$.

Alors,

Toute transition sortante en q' est dupliquée en une transition sortante en q .

Remarque

- Toute transition sortante au sens littéral, i.e on considère également la flèche de sortie/d'acceptation. Donc **si $q' \in F$, alors q doit également être un état final.**

5.5 Déterminisation

5.5.1 Existence et principe

Théorème - Existence d'un automate déterministe particulier

Soit A un automate non déterministe.

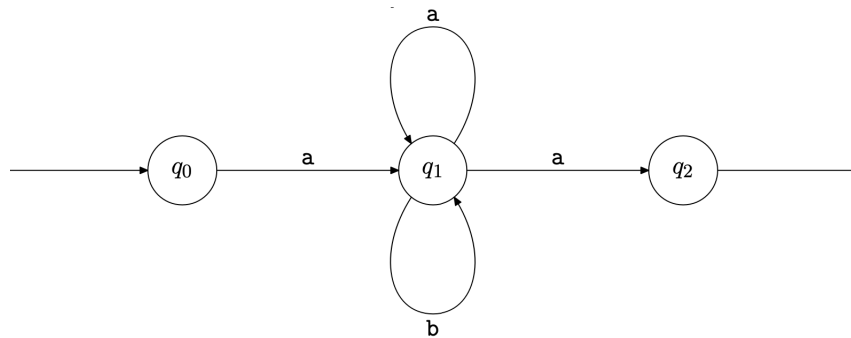
Alors,

il existe A' un automate déterministe reconnaissant le même langage.

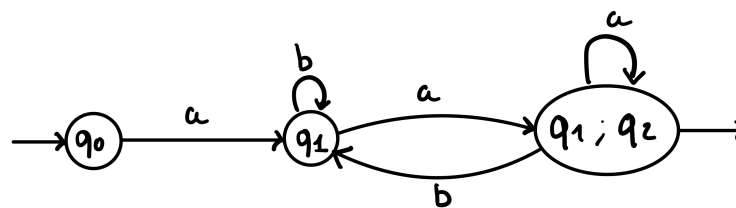
C'est la construction de l'automate des parties!

EXEMPLES. Déterminisons les automates suivants :

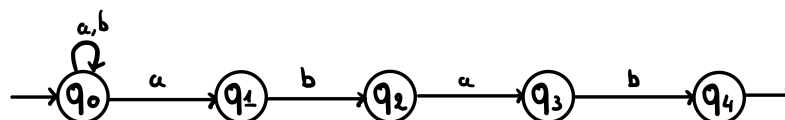
1. Soit donc l'automate



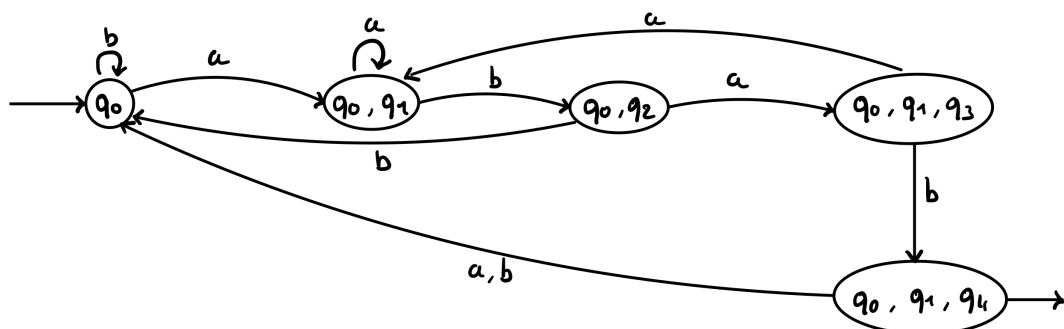
On a alors :



2. Ensuite, on dispose de l'automate :



En le déterminisant, on a :



DÉMONSTRATION.

On suppose disposer d'un AFND $A = (\Sigma, Q, I, F, \delta)$ sans ε -transition (on ajoutera plus tard une preuve qui les prendra en compte) et on construit :

$$\tilde{A} = (\Sigma, \mathcal{P}(Q), I, \{P \subset Q, P \cap F \neq \emptyset\}, \tilde{\delta})$$

$$\text{où } \forall P \subset Q, \forall a \in \Sigma, \tilde{\delta}(P, a) = \bigcup_{p \in P} \delta(p, a).$$

Il faut montrer par récurrence sur $|m|$ (avec $m \in \Sigma^*$) que pour tout $P \subset Q$, $\tilde{\delta}^*(P, m)$ est l'ensemble des états de Q pour lesquels il existe $p \in P$ tel que cet état soit dans $\delta^*(p, m)$, i.e $\tilde{\delta}^*(P, m) = \bigcup_{p \in P} \delta^*(p, m)$.

➤ On veut donc montrer que :

$$\forall P \subset Q, \forall m \in \Sigma^*, \tilde{\delta}^*(P, m) = \bigcup_{p \in P} \delta^*(p, m)$$

$$\text{➤ Si } m = \varepsilon \text{ alors } \tilde{\delta}^*(P, m) = P = \bigcup_{p \in P} \underbrace{\delta^*(p, m)}_{=\{p\}}$$

➤ Soit $n \in \mathbb{N}$ et soit $m = am'$ avec $a \in \Sigma$ et $m' \in \Sigma^n$. Alors :

$$\begin{aligned} \tilde{\delta}^*(P, m) &= \tilde{\delta}^*(P, am') \\ &= \tilde{\delta}^*(\tilde{\delta}(P, a), m') \\ &= \bigcup_{p \in \tilde{\delta}(P, a)} \delta^*(p, m') \text{ (par H.R)} \\ &= \bigcup_{p \in P} \underbrace{\bigcup_{q \in \delta(p, a)} \delta^*(q, m')}_{\delta^*(p, am')} \\ &= \bigcup_{p \in P} \delta^*(p, am') \text{ (par définition de } \delta^* \text{ dans le cas non-déterministe)} \end{aligned}$$

d'où l'hérédité.

Ainsi, pour $m \in \Sigma^*$,

$$\begin{aligned} m \in \mathcal{L}(\tilde{A}) &\Leftrightarrow \tilde{\delta}^*(I, m) \cap F \neq \emptyset \\ &\Leftrightarrow \exists i \in I \text{ et un chemin étiqueté par } m \text{ depuis } i \text{ qui mène à un état de } F \\ &\Leftrightarrow m \in \mathcal{L}(A) \end{aligned}$$

D'où le résultat voulu.

Enfin, si on dispose d'un automate avec des ε -transitions, on détermine directement en compte les ε -transitions.

5.5.2 Complexité

Théorème - Complexité de la détermination d'un automate

Soit $n \geq 1$ un entier naturel.

Il existe un AFND à $n + 1$ états, tel que tout AFD reconnaissant le même langage ait au moins 2^n états.

La détermination est donc un algorithme de coût exponentiel.

DÉMONSTRATION.

Pour $n \in \mathbb{N}$, on définit $L_n = \Sigma^* a \Sigma^{n-1}$. On considère A un AFND tq. $\mathcal{L}(A) = L_n$.

Soit $A_d = (Q, q_0, F, \delta)$ un AFD qui reconnaît L_n . Soient $u, v \in \Sigma^n$ tq. $u \neq v$.

Soit w le plus long suffixe commun à u et v . Sans perte de qualité, il existe $u', v' \in \Sigma^*$ tq $\begin{cases} u = u'aw \\ v = v'bw \end{cases}$ avec $a \neq b$.

On pose $\tilde{w} = b^{n-1-|w|}$ et on remarque que :

$$\begin{aligned} u\tilde{w} &= u' a \underbrace{wb^{n-1-|w|}}_{n-1 \text{ lettres}} \in L_n \\ v\tilde{w} &= v' bwb^{n-1-|w|} \notin L_n \end{aligned}$$

Ainsi, si $\text{Card}(Q) < 2^n$ alors $\exists u \neq v \in \Sigma^n$ tq. $\delta^*(q_0, u) = \delta^*(q_0, v)$ (principe des tiroirs).

Et donc, par construction de $\tilde{w}$, on a

$$\begin{aligned}\delta^*(q_0, u\tilde{w}) &= \delta^*(\delta^*(q_0, u), \tilde{w}) \\ &= \delta^*(\delta^*(q_0, v), \tilde{w}) \\ &= \delta^*(q_0, v\tilde{w})\end{aligned}$$

or $u\tilde{w} \in L_n$ et $v\tilde{w} \in L_n$.
D'où l'absurdité.

Méthode : On a mis en évidence une famille F de mots (ici Σ^n) tq. $m \in F \mapsto \delta^*(q_0, m) \in Q$ est injective. On peut alors en déduire que $|Q| \geq |F|$.

Pour construire F , il faut des mots $u, v \in F$ tq. $u \neq v, \exists w \in \Sigma^*$ tq $uw \in L$ alors que $vw \notin L$.

On dira de deux automates qu'ils sont équivalents s'ils reconnaissent le même langage. On a vu que tout automate non déterministe (sans ϵ -transitions) est équivalent à

- * un automate déterministe;
- * un automate déterministe complet (sans blocage);
- * un automate déterministe émondé;
- * un automate déterministe standardisé.

5.6 Bilan

- On appelle *langage reconnaissable* un langage pour lequel il existe un AFD le reconnaissant.
- L est dit *reconnaissable* ssi il est reconnu par un AFND.
- L est *reconnaissable* ssi il est reconnu par un AFND avec ϵ -transitions.

Théorème de Kleene

Un langage est reconnaissable si et seulement si il est régulier.

6 Les langages réguliers sont inclus dans les langages reconnaissables

6.1 Automate de Thompson

Pour rappel, la classe des langages réguliers est la plus petite classe de langages contenant $\emptyset, \{a\}$ pour tout $a \in \Sigma$ et qui est stable par union, concaténation et étoile de Kleene.

Définition - REG et REC

On note,

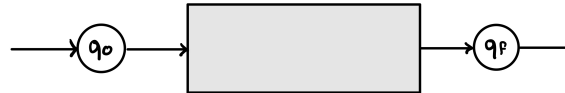
REG l'ensemble des langages réguliers.
REC l'ensemble des langages reconnaissables.

En fait, pour montrer que $\text{REG} \subset \text{REC}$, il suffit de montrer que $\emptyset \in \text{REC}, \forall a \in \Sigma, \{a\} \in \text{REC}$ et que REC est stable par union, concaténation et étoile de Kleene car la minimalité de REG nous garantirait l'inclusion recherchée.

Définition - Automate standardisé

Un automate fini non-déterministe est dit **standardisé** s'il dispose d'un unique état initial sans flèche entrante et d'un unique état final sans flèche sortante.

En somme, il ressemble à cela :



Théorème

Tout AFND est équivalent à un automate standardisé.

DÉMONSTRATION.

Soit $A = (\Sigma, Q, I, F, \delta, \varphi)$. On définit :

$$A_S = (\Sigma, Q \cup \{q_i, q_f\}, \{q_i\}, \{q_f\}, \delta_S, \varphi_S)$$

avec $\varphi_S(q_i) = I$ et $\forall f \in F, \varphi_S(f) = \varphi(f) \cup \{q_f\}$.



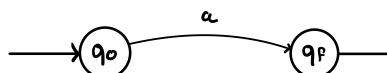
On conclut en montrant (péniblement) que $\mathcal{L}(A) = \mathcal{L}(A_S)$.

EXEMPLES.

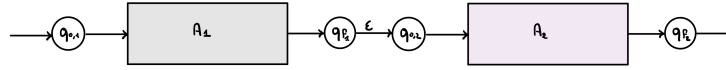
➤ Automate standardisé reconnaissant $\emptyset$:



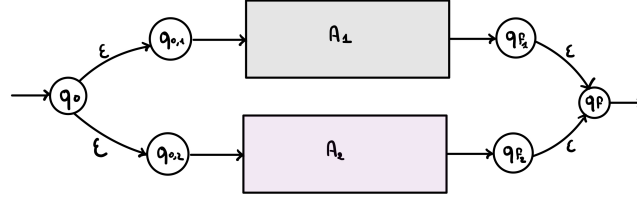
➤ Automate standardisé qui reconnaît $\{a\}$ pour $a \in \Sigma$:



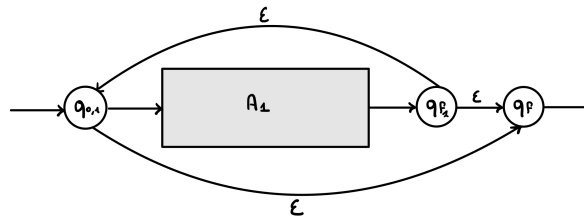
➤ Concaténation :



➤ Union :



➤ Étoile :



6.2 Algorithme de Berry-Sethi

Définition - Langage local

Soit L un langage. L est dit **local** ssi :

$$\exists P, S \subset \Sigma \text{ et } N \subset \Sigma^2 \text{ tq. } L \setminus \{\varepsilon\} = (P\Sigma^*S) \setminus (\Sigma^*N\Sigma^*)$$

On remarque que :

- P représente toutes les lettres par lesquelles les mots de L peuvent commencer.
- S représente toutes les lettres par lesquelles les mots de L peuvent terminer.
- N représente les facteurs de 2 lettres interdits dans les mots de L .

EXEMPLE. Pour $L = a + (ab)^*$, on a

- $P = \{a\}$
- $S = \{a; b\}$
- $F = \Sigma^2 \setminus N = \{ab; ba\}$

on remarque alors que $aba \in (P\Sigma^*S) \setminus (\Sigma^*N\Sigma^*)$ mais $aba \notin L$.

Théorème

Tout langage local est reconnaissable.

DÉMONSTRATION.

On prend P, S et $F = \Sigma^2 \setminus N$. On définit :

$$A = (\Sigma, \{q_i\} \cup \{q_a \mid a \in \Sigma\}, q_i, \{q_a \mid a \in S\}, \delta)$$

$$\text{avec } \begin{cases} \forall a \in P, \delta(q_i, a) = q_a \\ \forall ab \in F, \delta(q_a, b) = q_b \end{cases}$$

Ainsi, $m \in \mathcal{A}$ ssi il commence par un élément de P et contient uniquement des facteurs de taille 2 dans F et dont la dernière lettre appartient à S .

d'où le résultat voulu.

Remarques

- Si L contient ε , il faut rajouter q_i aux états acceptants.
- Un automate qui vérifie que $\forall q \in Q, \exists! a \in \Sigma$ tq. toutes les transitions étiquetées par a aboutissent dans q est dit local.

Proposition

Un langage est local ssi il est reconnu par un automate local.

Définition - Expression régulière linéaire

Une expression régulière est dite **linéaire** ssi chaque symbole de lettres est utilisé une seule fois.

Théorème

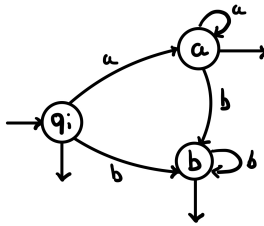
Tout langage dénoté par une expression linéaire est local.

EXEMPLES.

1. Prenons $L = a^*b^*$. On a alors :

$$\begin{cases} P = \{a; b\} \\ S = \{a; b\} \\ F = \{aa; ab; bb\} \end{cases}$$

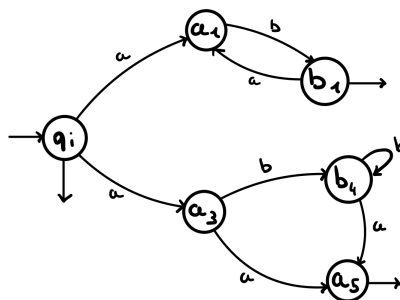
et on obtient l'automate suivant :



2. Prenons $L = (ab)^* + ab^*a$. On linéarise alors l'expression : $L = (a_1b_2)^* + a_3b_4^*a_5$. On a alors :

$$\begin{cases} P = \{a_1, a_3\} \\ S = \{b_1, a_5\} \\ F = \{a_1b_1, b_4b_4, b_1a_1, a_3a_5, a_3b_4, b_4a_5\} \end{cases}$$

On obtient alors l'automate suivant :



3. Prenons $L = aa(ab)^*b$. Comme précédemment, on linéarise l'expression : $L = a_1a_2(a_3|a_4b_5)^*b_6$. On a alors :

$$\begin{cases} P = \{a_1\} \\ S = \{b_6\} \\ F = \{a_1a_2, a_2a_3, a_2a_4, a_2b_6, a_3a_3, a_3a_4, a_3b_6, a_4b_6, a_4b_5, b_5a_3, b_5a_4, b_5b_6\} \end{cases}$$

On en déduit l'automate de Glushkov résultant de P, S et F (exo).

Exercice

Écrire des fonctions en Caml qui prennent en entrée une expression régulière et renvoie P, S, F (sous forme de listes).

```
1 type e = Vide
2   | Epsilon
3   | Lettre of char
4   | Union of e*e
5   | Concat of e*e
6   | Etoile of e
7
8 let rec P e =
9   match e with
10  | Vide-> [], false
11  | Epsilon -> [], true
12  | Lettre(a) -> [a], false
13  | Union(e1,e2) -> let l1,b1 = P e1 in
14                    let l2,b2 = P e2 in
15                    l1 @ l2 , b1 || b2
16  | Concat(e1,e2) -> let l1,b1 = P e1 in
17                    let l2,b2 = P e2 in
18                    (if b1 then l1 @ l2 else l1), b1 && b2
19  | Etoile(e1) -> let l,b = P e1 in l, true
```

On se rend vite compte (en écrivant la fonction $F : e \rightarrow \text{char list}$) qu'il faut faire appel à P et S , on va donc créer une fonction qui renvoie P, S et F selon son propre appel récursif (et pas ceux d'une autre fonction).

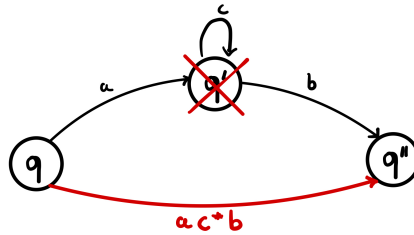
```
1 let rec T e =
2   match e with
3   | Vide -> [], [], [], false
4   | Epsilon -> [], [], [], true
5   | Lettre(a) -> [a], [a], [], false
6
7   | Union(e1,e2) -> let lp1,ls1,lf1,b1 = T e1 in
8                     let lp2,ls2,lf2,b2 = T e2 in
9                     lp1 @ lp2, ls1 @ ls2, lf1 @ lf2, b1 || b2
10
11  | Concat(e1,e2) -> let lp1,ls1,lf1,b1 = T e1 in
12                    let lp2,ls2,lf2,b2 = T e2 in
13                    (if b1 then lp1 @ lp2 else lp2), (if b2 then ls1 @ ls2 else ls2), lf1 @ lf2 @ (produit ls1 lp2),
14                    b1 && b2
15
16  | Etoile(e1) -> let lp1,ls1,lf1,b1 = T e1 in lp1,ls1,lf1 @ (produit ls1 lp1), true
```

On dispose donc d'une fonction $\text{produit} : \text{char list} \rightarrow \text{char list} \rightarrow \text{string list}$ qui réalise le produit cartésien de deux ensembles de char , en réalisant toutes les combinaisons possibles donc (elles deviennent donc des string de longueur 2).

7 Élimination des états

Objectif : Trouver une expression régulière dénotant le langage reconnu par un automate.

➤ En général, l'expression obtenue est inutilement compliquée.

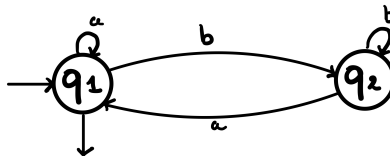


⇒ On va donc étiqueter les transitions par des expressions.

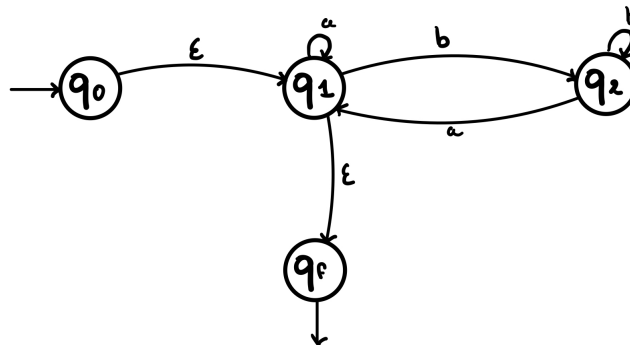
Méthode d'élimination des transitions

- **1ère étape : On standardise l'automate!!** (i.e on fait en sorte d'avoir un unique état initial et un unique état final sans flèche entrante et le final sans sortante)
- **2ème étape :** On élimine un à un tous les états sauf l'initial et le final pour obtenir juste une transition entre ces deux états avec comme étiquette e l'expression recherchée.

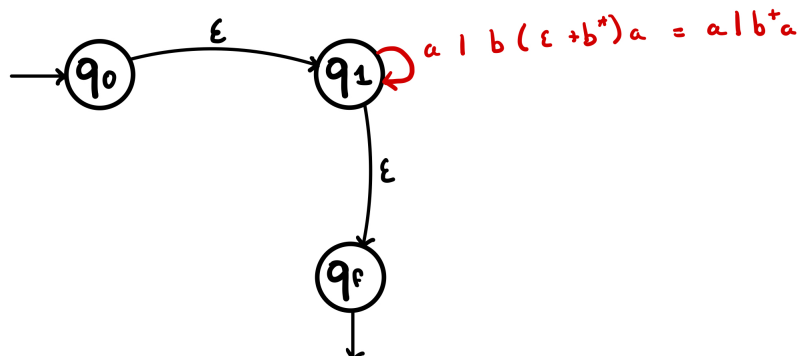
EXEMPLE. Appliquons notre méthode à l'automate suivant :



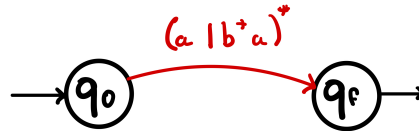
➤ **1ère étape :**



➤ **2ème étape :** On élimine d'abord q_2



Puis on élimine q_1 :



Élimination générale d'un état q

On identifie :

$$E_q = \{q' \in Q, q' \neq q, \exists e \in \text{REG tq. } \delta(q', e) = q\}$$

$$S_q = \{q' \in Q, q' \neq q, \exists a \in \text{REG tq. } \delta(q, a) = q'\}$$

et donc $\forall (q', q'') \in E_q \times S_q$, on met une arête de q' vers q'' étiquetée par $ef^*g|h$ avec :

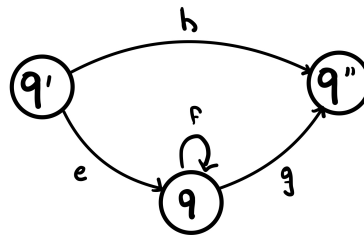
e l'étiquette de $q' \rightarrow q$

f l'étiquette de $q \rightarrow q$

g l'étiquette de $q \rightarrow q''$

h l'étiquette de $q' \rightarrow q''$

Tout simplement, on passe d'une situation comme celle exposée ci-dessous :

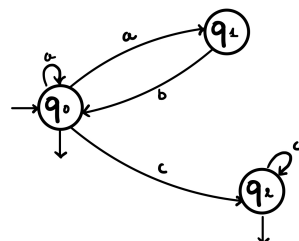


à la situation suivante :



Exercice

Éliminer les états de l'automate ci-dessous pour en déduire le langage reconnu.



On arrive à $L = (ab|a)^*(c^+|\epsilon) = (ab|a)^*c^*$

8 Lemme de l'étoile

➤ Méthode pour montrer qu'un langage n'est pas reconnaissable (i.e non-régulier).

➤ Il faut avoir conscience qu'il s'agit seulement d'une condition nécessaire pour être reconnaissable.

EXEMPLE.

Prenons $L = \{a^n b^n \mid n \in \mathbb{N}\}$.

Supposons par l'absurde que L soit reconnaissable. Soit donc $A = (\Sigma, Q, q_0, F, \delta)$ un AFD complet à N états.

Soit $m = a^N b^N$. Pour tout $i \in \llbracket 0; N \rrbracket$, notons $q_i = \delta^*(q_0, a^i)$.

Par principe des tiroirs, il existe j tq. $i < j \leq N$ et $q_i = q_j$.

Ainsi,

$$\underbrace{\delta^*(q_0, a^i a^{N-j} b^N)}_{\notin F \text{ car } N+i-j \neq N} = \underbrace{\delta^*(q_0, a^j a^{N-j} b^N)}_{\in F}$$

ce qui est bel et bien absurde.

Théorème - Lemme de l'étoile

Si L est reconnaissable alors :

$$\exists N \in \mathbb{N} \text{ tq. } \forall m \in L, |m| \geq N, \exists u, v, w \in \Sigma^* \text{ tq. } \begin{cases} m = uvw \\ |uv| \leq N \\ v \neq \varepsilon \end{cases} \text{ et } \forall k \in \mathbb{N}, uv^k a \in L$$

En pratique on utilisera la contraposée : Si

$$\forall N \in \mathbb{N}, \exists m \in L, |m| \geq N, \forall u, v, w \in \Sigma^* \text{ tq. } \begin{cases} m = uvw \\ |uv| \leq N \\ v \neq \varepsilon \end{cases} \text{ et } \exists k \in \mathbb{N}, uv^k a \notin L$$

alors L n'est pas reconnaissable.

DÉMONSTRATION.

Soit L un langage reconnaissable et soit $A = (\Sigma, Q, q_0, F, \delta)$ un AFD complet tq. $\mathcal{L}(A) = L$. Soit $N \in \mathbb{N}$ le nombre d'états de A .

Soit $m \in L$ tq. $|m| \geq N$. Notons $m = m_1 \dots m_k$ et $\forall i \in \llbracket 0; k \rrbracket$, posons $q_i = \delta^*(q_0, m_1 \dots m_i)$.

Par principe des tiroirs ($k \geq N$) : $\exists i, j$ tq. $0 \leq i < j \leq k$ et $q_i = q_j$.

On pose alors $\underbrace{u = m_1 \dots m_i}_{q_0 \xrightarrow{u} q_i}, \underbrace{v = m_{i+1} \dots m_j}_{q_i \xrightarrow{v} q_j = q_i} \text{ et } \underbrace{w = m_{j+1} \dots m_k}_{q_i = q_j \xrightarrow{w} q_f}$.

On a bien sûr $m = uvw$ mais aussi $v \neq \varepsilon$ (car $i < j$) et $|uv| = j \leq N$.

Ensuite, prouvons par récurrence sur k que $\delta^*(q_i, v^k) = q_i$:

- $k = 0$: on a $v^0 = \varepsilon$ et donc on a bien $\delta^*(q_i, v^0) = \delta^*(q_i, \varepsilon) = q_i$
- $k = 1$: on a (par construction) $\delta^*(q_i, v^1) = \delta^*(q_i, v) = q_j = q_i$
- Hérédité : Soit $k \geq 0$, on a

$$\begin{aligned} \delta^*(q_i, v^{k+1}) &= \delta^*(\underbrace{\delta^*(q_i, v^k)}_{= q_i \text{ par H.R.}}, v) \\ &= \delta^*(q_i, v) \\ &= q_j = q_i \end{aligned}$$

Enfin, soit $k \in \mathbb{N}$,

$$\begin{aligned} \delta^*(q_0, uv^k w) &= \delta^*(\delta^*(q_0, u), v^k w) \\ &= \delta^*(q_0, v^k w) \\ &= \delta^*(\delta^*(q_i, v^k), w) \\ &= \delta^*(q_i, w) \\ &= q_f \in F \end{aligned}$$

EXEMPLE. Prenons donc le langage $\{a^n b^n \mid n \in \mathbb{N}\}$ Soit $N \in \mathbb{N}$.

Soit $m \in L$ tq. $m = a^N b^N$.

Soient u, v, w quelconques tq.
$$\begin{cases} a^N b^N = uvw \\ |uv| \leq N \\ v \neq \varepsilon \end{cases}$$

Alors $\exists i \in [0; N-1], j > 0$ avec $i + j \leq N$ tq. $u = a^i, v = a^j, w = a^{N-i-j} b^N$.

Enfin, pour $k = 2$, on a

$$uv^k w = a^i a^{2j} a^{N-i-j} b^N = a^{N+j} b^N \notin L \text{ (car } j > 0)$$

Donc d'après la contraposée du lemme de l'étoile, L n'est pas reconnaissable. D'où le résultat voulu.

Exercice

Montrer que les langages suivants ne sont pas réguliers :

- $L_1 = \{w \bar{w} \mid w \in \Sigma^*\}$ où $\bar{w}$ est le mot miroir de w .
- $L_2 = \{a^n b a^n \mid n \in \mathbb{N}\}$
- $L_3 = \{a^n b^p \mid n \geq p \in \mathbb{N}\}$
- $L_4 = \{a^{n^2} \mid n \in \mathbb{N}\}$
- $L_5 = \{a^p \mid p \in \mathbb{P}\}$

- Soit donc $L_1 = \{w \bar{w} \mid w \in \Sigma^*\}$ où $\bar{w}$ est le mot miroir de w .

Soit $N \in \mathbb{N}$, on pose $m = a^N b b a^N \in L_1$

Soient u, v, w tq. $m = uvw$ avec $|uv| \leq N, v \neq \varepsilon$, i.e $\exists i, j \in [0; N]$ tq. $i + j \leq N, j > 0$ et
$$\begin{cases} u = a^i \\ v = a^j \\ w = a^{N-i-j} b b a^N \end{cases}$$

On a $uw \notin L_1 (k=0)$ et $uv^2 w \notin L_1 (k=2)$, ce qui, d'après la contraposée du lemme de l'étoile, nous permet de conclure.

- Soit donc $L_2 = \{a^n b a^n \mid n \in \mathbb{N}\}$.

Soit $N \in \mathbb{N}$, on pose $m = a^N b a^N \in L_2$ et on conclut comme précédemment.

- Soit donc $L_3 = \{a^n b^p \mid n \geq p \in \mathbb{N}\}$.

Soit $N \in \mathbb{N}$, on pose $m = a^N b^N \in L_3$.

Soient u, v, w tq. $m = uvw$ avec $|uv| \leq N, v \neq \varepsilon$, i.e $\exists i, j \in [0; N]$ tq. $i + j \leq N, j > 0$ et
$$\begin{cases} u = a^i \\ v = a^j \\ w = a^{N-i-j} b^N \end{cases}$$

Pour $k=0, uw = a^{N-j} b^N \notin L_3$ car $j > 0$, ce qui nous permet de conclure.

- Soit donc $L_4 = \{a^{n^2} \mid n \in \mathbb{N}\}$.

Soit $N \in \mathbb{N}$, on pose $m = a^{N^2} \in L_4$.

Soient u, v, w tq. $m = uvw$ avec $|uv| \leq N, v \neq \varepsilon$, i.e $\exists i, j \in [0; N]$ tq. $i + j \leq N, j > 0$ et
$$\begin{cases} u = a^i \\ v = a^j \\ w = a^{N^2-i-j} \end{cases}$$

En prenant $k=2, uv^2 w = a^{i+2j+N^2-i-j} = a^{N^2+j}$. Or $j > 0$ donc $N^2 + j > N^2$ et $j \leq N$ donc $N^2 + j \leq N^2 + N < (N+1)^2$.

Donc $N^2 + j$ ne peut pas être un carré donc $a^{N^2+j} = uv^2 w \notin L_4$, ce qui nous permet de conclure.

- Soit donc $L_5 = \{a^p \mid p \in \mathbb{P}\}$

Soit $N \in \mathbb{N}$, on pose $m = a^p \in L_5$ avec $p \in \mathbb{P}$ tq. $p \geq N$.

Soient u, v, w tq. $m = uvw$ avec $|uv| \leq N, v \neq \varepsilon$, i.e $\exists i, j \in [0; N]$ tq. $i + j \leq N, j > 0$ et
$$\begin{cases} u = a^i \\ v = a^j \\ w = a^{p-i-j} b^N \end{cases}$$

En prenant $k = p + 1$,

$$\begin{aligned} uv^k w &= a^{i+j(p+1)+p-i-j} \\ &= a^{p+jp} \\ &= a^{p(1+j)} \notin L_5 \text{ car } 1+j > 1 \text{ et donc } p(j+1) \notin P \end{aligned}$$

ce qui permet de conclure quand à la non-régularité de ce langage.